

Modül 1: SQL Temelleri ve Derin Bakış (Comprehensive Foundations)

Bir veri bilimci için SQL, veritabanından veri "çekmekten" ziyade, veriyi filtreleme, temizleme ve ilk keşifsel analiz (EDA) sürecidir.

1.1. SQL ve İlişkisel Veritabanı (RDBMS) Mantığı

Veri biliminde veriler genellikle tablolar halinde saklanır.

- **RDBMS (İlişkisel Veritabanı Yönetim Sistemi):** Verilerin birbirleriyle ilişkili tablolar setinde saklandığı sistemdir (PostgreSQL, MySQL, SQL Server).
- **Tablo Yapısı: Sütunlar (Features/Özellikler) ve Satırlar (Observations/Gözlemler)** birleşerek veri setini oluşturur.

1.2. Veri Tipleri: Veri Bilimcinin Pusulası

Analiz yapmadan önce sütunun hangi tipte olduğunu bilmek zorunludur. Yanlış veri tipi, yanlış hesaplama demektir.

- **Sayısal:** INT, FLOAT, DECIMAL (Finansal hassasiyet için).
- **Metinsel:** VARCHAR (Esnek uzunluk), CHAR (Sabit uzunluk).
- **Tarih:** DATE, TIMESTAMP (Zaman serisi analizleri için).
- **Mantıksal:** BOOLEAN (1/0 veya True/False).

1.3. Temel Sorgu Yapısı: SELECT, FROM ve ALIAS

- **SELECT:** Sütunları seçer. Tüm sütunlar için **SELECT *** kullanılır.
- **FROM:** Kaynak tabloyu belirtir.
- **AS (Alias):** Sütunlara veya tablolara geçici isimler verir. Kodun okunabilirliğini artırır ve *Feature Engineering* için kullanılır.

Örnek:

```
SELECT
    isim,
    maas AS brut_maas,
    maas * 0.85 AS net_maas
FROM calisanlar;
```

1.4. Benzersiz Kayıtlar: DISTINCT

Veri setindeki tekrar eden satırları temizlemek veya kategorik bir sütundaki sınıfları görmek için kullanılır.

```
SELECT DISTINCT şehir FROM musteriler; -- Kaç farklı şehir olduğunu listeler.
```

1.5. Veri Filtreleme: WHERE Yan Cümlecığı ve Operatörler

Veri biliminde sadece ilgilendiğimiz alt küme (subset) ile çalışırız.

A. Karşılaştırma Operatörleri

- =, <>, !=, <, >, <=, >=

B. Mantıksal Operatörler (AND, OR, NOT)

Birden fazla koşulu birleştirir.

Kritik Bilgi (İşlem Sırası): SQL önce AND operatörünü çalıştırır. Karışıklığı önlemek için mutlaka **parantez** kullanın.

```
SELECT * FROM urunler
WHERE (kategori = 'Kitap' OR kategori = 'Hobi') AND fiyat < 100;
```

1.6. Aralık, Liste ve NULL Kontrolü

- **BETWEEN:** Belirli bir aralıktaki değerleri getirir (Sınırlar dahildir).
- **IN:** Bir sütunun değerinin liste içinde olup olmadığını kontrol eder.
- **IS NULL / IS NOT NULL:** Boş (bilinmeyen) değerleri bulmak için tek yoldur. (= NULL çalışmaz!)

1.7. Metin Arama: LIKE ve Pattern Matching

Veri temizleme aşamasında anahtar kelime aramak için kullanılır.

- **% :** Sıfır veya daha fazla karakter.
- **_ :** Sadece tek bir karakter.

```
WHERE email LIKE '%@gmail.com' -- Gmail adreslerini bulur.
WHERE telefon LIKE '5__-%' -- 5 ile başlayan ve belirli formatta olanları bulur.
```

1.8. Sıralama ve Sınırlama: ORDER BY ve LIMIT

- **ORDER BY:** Veriyi belirli bir sütuna göre dizer. ASC (Artan-Varsayılan) veya DESC (Azalan).
- **LIMIT:** Sadece belirli sayıda satırı döndürür. Milyonlarca satırlık veride sadece ilk 5 satıra bakmak (Head) için kullanılır.

```
SELECT * FROM satislar
ORDER BY tarih DESC, tutar ASC
LIMIT 5;
```

1.9. İleri Seviye: SQL Mantıksal Çalışma Sırası

Bir SQL motoru, kodunuzu yazdığınız sırayla okumaz. Bu sırayı bilmek, hata ayıklarken (debugging) hayat kurtarır:

1. **FROM / JOIN:** Önce veri kaynağına gidilir.
2. **WHERE:** Satırlar filtrelendir.
3. **GROUP BY:** Gruplama yapılır.
4. **HAVING:** Gruplanmış veriler filtrelendir.
5. **SELECT:** Sütunlar seçilir (Alias burada atanır).
6. **ORDER BY:** Sıralama yapılır.
7. **LIMIT:** Sonuç kısıtlanır.

Veri Bilimciler İçin Notlar:

1. **Performans:** SELECT * yerine sadece ihtiyacın olan sütunları seçmek hızı artırır.
2. **Okunabilirlik:** Karmaşık WHERE koşullarında parantez kullanmayı alışkanlık edinin.
3. **Veri Keşfi:** Yeni bir tabloya bakarken her zaman LIMIT 10 ile verinin tipine ve içeriğine göz atın.